Ex.No:1 Write a code simulating ARP /RARP protocols.

Aim

To develop a Java program that simulates the working of **ARP (Address Resolution Protocol)** and **RARP (Reverse Address Resolution Protocol)** by mapping IP addresses to MAC addresses and vice versa.

Algorithm

ARP (IP Address → MAC Address)

1. Start the program.
2. Create an ARP table containing IP addresses and corresponding MAC addresses.
3. Display the menu options.
4. Accept an IP address from the user.
5. Search the ARP table for the entered IP address.
6. If found, display the corresponding MAC address.
7. Otherwise, display "IP Address not found".
8. Return to the menu.

RARP (MAC Address → IP Address)

1. Accept a MAC address from the user.
2. Search the ARP table for the entered MAC address.
3. If found, display the corresponding IP address.
4. Otherwise, display "MAC Address not found".
5. Return to the menu.
6. Stop when the user chooses Exit.

Program

```
import java.util.HashMap;
import java.util.Scanner;

public class ARPRARPSimulation {

   public static void main(String[] args) {

      HashMap<String, String> arpTable = new HashMap<>();

      arpTable.put("192.168.1.1", "AA:BB:CC:DD:EE:01");
      arpTable.put("192.168.1.2", "AA:BB:CC:DD:EE:02");
      arpTable.put("192.168.1.3", "AA:BB:CC:DD:EE:03");

      Scanner sc = new Scanner(System.in);

      while (true) {
```

```
System.out.println("\n----- ARP / RARP Simulation -----");
System.out.println("1. ARP (IP to MAC)");
System.out.println("2. RARP (MAC to IP)");
System.out.println("3. Exit");
System.out.print("Enter your choice: ");

int choice = sc.nextInt();
sc.nextLine();

switch (choice) {

   case 1:
      System.out.print("Enter IP Address: ");
      String ip = sc.nextLine();

      if (arpTable.containsKey(ip)) {
         System.out.println("MAC Address: "
               + arpTable.get(ip));
      } else {
         System.out.println("IP Address not found.");
      }
      break;

   case 2:
      System.out.print("Enter MAC Address: ");
      String mac = sc.nextLine();

      boolean found = false;

      for (String key : arpTable.keySet()) {
         if (arpTable.get(key)
               .equalsIgnoreCase(mac)) {

            System.out.println("IP Address: " + key);
            found = true;
            break;
         }
      }

      if (!found) {
         System.out.println("MAC Address not found.");
      }
      break;

   case 3:
      System.out.println("Exiting...");
      sc.close();
      System.exit(0);
```

```
default:

System.out.println("Invalid Choice!");

 }       }

  }           }
```

Sample Output

Output 1: ARP Operation

```
----- ARP / RARP Simulation -----
1. ARP (IP to MAC)
2. RARP (MAC to IP)
3. Exit

Enter your choice: 1
Enter IP Address: 192.168.1.2

MAC Address: AA:BB:CC:DD:EE:02
```

Output 2: RARP Operation

```
----- ARP / RARP Simulation -----
1. ARP (IP to MAC)
2. RARP (MAC to IP)
3. Exit

Enter your choice: 2
Enter MAC Address: AA:BB:CC:DD:EE:03

IP Address: 192.168.1.3
```

Output 3: Exit

```
----- ARP / RARP Simulation -----
1. ARP (IP to MAC)
2. RARP (MAC to IP)
3. Exit

Enter your choice: 3

Exiting...
```

Result

Thus, the Java program to simulate **ARP (Address Resolution Protocol)** and **RARP (Reverse Address Resolution Protocol)** was implemented successfully, and the corresponding MAC address for a given IP address and IP address for a given MAC address were obtained.

Ex.No:2 Write a HTTP web client program to download a web page using TCP sockets

Aim

To develop a Java program that acts as an HTTP web client and downloads a web page from a web server using TCP sockets.

Algorithm

1. Start the program.
2. Specify the web server hostname and port number (default HTTP port is 80).
3. Create a TCP socket connection to the web server.
4. Obtain input and output streams from the socket.
5. Send an HTTP GET request to the server.
6. Read the server's response through the input stream.
7. Display the downloaded web page content.
8. Close the socket connection.
9. Stop the program.

Program

```
import java.io.*;
import java.net.*;

public class HTTPWebClient {

   public static void main(String[] args) {
      try {
         // Connect to the web server
         Socket socket = new Socket("example.com", 80);

         // Output stream to send HTTP request
         PrintWriter out = new PrintWriter(
               socket.getOutputStream(), true);

         // Input stream to receive response
         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));

         // Send HTTP GET request
         out.println("GET / HTTP/1.1");
         out.println("Host: example.com");
         out.println("Connection: close");
         out.println();

         String line;




System.out.println("Downloaded Web Page:\n");

         // Read and display server response
```

```
            while ((line = in.readLine()) != null) {
                System.out.println(line);
            }

            // Close resources
            in.close();
            out.close();
            socket.close();

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Sample Output

Downloaded Web Page:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256
Connection: close

<!doctype html>
<html>
<head>
    <title>Example Domain</title>
</head>
<body>
<div>
    <h1>Example Domain</h1>
    <p>This domain is for use in illustrative examples.</p>
</div>
</body>
</html>
```

Result

Thus, the HTTP web client program was successfully implemented using Java TCP sockets, and the web page content was downloaded and displayed from the web server.

Ex.No:3 Simulation of DNS using UDP sockets.

Aim

To develop a Java program that simulates the working of a **Domain Name System (DNS)** using **UDP sockets**, where a client sends a domain name to the server and receives the corresponding IP address.

Algorithm

DNS Server

1. Start the server.
2. Create a DatagramSocket and bind it to a port number.
3. Maintain a DNS table containing domain names and their corresponding IP addresses.
4. Wait for a client request.
5. Receive the domain name from the client.
6. Search for the domain name in the DNS table.
7. If found, retrieve the corresponding IP address.
8. Otherwise, return "Domain not found".
9. Send the response to the client using a UDP packet.
10. Repeat Steps 4–9.

DNS Client

1. Start the client.
2. Create a DatagramSocket.
3. Read the domain name from the user.
4. Send the domain name to the DNS server using a UDP packet.
5. Receive the response from the server.
6. Display the IP address received.
7. Close the socket.
8. Stop the program.

Program

DNS Server

```
import java.net.*;
import java.util.HashMap;

public class DNSServer {
    public static void main(String[] args) {
        try {
            DatagramSocket serverSocket = new DatagramSocket(9876);



            HashMap<String, String> dnsTable = new HashMap<>();

            dnsTable.put("google.com", "142.250.190.78");
            dnsTable.put("yahoo.com", "98.137.11.163");
            dnsTable.put("facebook.com", "157.240.22.35");
```

```
            dnsTable.put("amazon.com", "205.251.242.103");

            byte[] receiveData = new byte[1024];
            byte[] sendData;

            System.out.println("DNS Server is Running...");

            while (true) {

               DatagramPacket receivePacket =
                     new DatagramPacket(receiveData,
                           receiveData.length);

               serverSocket.receive(receivePacket);

               String domain =
                     new String(receivePacket.getData(), 0,
                           receivePacket.getLength());

               System.out.println("Requested Domain: " + domain);

               String ip =
                     dnsTable.getOrDefault(domain,
                           "Domain Not Found");

               sendData = ip.getBytes();

               InetAddress clientAddress =
                     receivePacket.getAddress();

               int clientPort =
                     receivePacket.getPort();

               DatagramPacket sendPacket =
                     new DatagramPacket(sendData,
                           sendData.length,
                           clientAddress,
                           clientPort);

               serverSocket.send(sendPacket);
            }



} catch (Exception e) {
      System.out.println(e);   }   }  }
```

DNS Client

```
import java.net.*;
import java.util.Scanner;

public class DNSClient {
   public static void main(String[] args) {
      try {

         DatagramSocket clientSocket =
               new DatagramSocket();

         Scanner sc = new Scanner(System.in);

         System.out.print("Enter Domain Name: ");
         String domain = sc.nextLine();

         byte[] sendData = domain.getBytes();

         InetAddress serverAddress =
               InetAddress.getByName("localhost");

         DatagramPacket sendPacket =
               new DatagramPacket(sendData,
                     sendData.length,
                     serverAddress,
                     9876);
         clientSocket.send(sendPacket);

         byte[] receiveData = new byte[1024];

         DatagramPacket receivePacket =
               new DatagramPacket(receiveData,
                     receiveData.length);
         clientSocket.receive(receivePacket);

         String ip =
               new String(receivePacket.getData(),
                     0,
                     receivePacket.getLength());

         System.out.println("IP Address: " + ip);


clientSocket.close();
         sc.close();

      } catch (Exception e) {
         System.out.println(e);
      }   }
}
```

Sample Output
Server Output

DNS Server is Running...

Requested Domain: google.com
Requested Domain: yahoo.com
Client Output 1
Enter Domain Name: google.com

IP Address: 142.250.190.78
Client Output 2
Enter Domain Name: yahoo.com

IP Address: 98.137.11.163
Client Output 3
Enter Domain Name: twitter.com

IP Address: Domain Not Found

Result

Thus, the simulation of the **Domain Name System (DNS)** using **UDP sockets in Java** was implemented successfully..

Ex.No:4 Study of TCP/UDP performance using Simulation tool.

Aim

To study and compare the performance of **TCP (Transmission Control Protocol)** and **UDP (User Datagram Protocol)** by simulating data transmission between client and server using Java socket programming.

Algorithm

TCP Performance Study

1. Start the TCP server and bind it to a port number.
2. Wait for the client connection.
3. Establish a connection between the client and server.
4. Send a message from the client to the server.
5. Record the start and end time of transmission.
6. Calculate the transmission time.
7. Display the received message and transmission time.
8. Close the connection.

UDP Performance Study

1. Start the UDP server and bind it to a port number.
2. Wait for a datagram packet from the client.
3. Send a message from the client to the server.
4. Record the start and end time of transmission.
5. Calculate the transmission time.
6. Display the received message and transmission time.
7. Close the sockets.

Program

TCP Server

```
import java.io.*;
import java.net.*;

public class TCPServer {
   public static void main(String[] args) {
      try {
         ServerSocket server = new ServerSocket(5000);
         System.out.println("TCP Server Waiting...");

         Socket socket = server.accept();

         long startTime = System.nanoTime();


        BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));

         String message = in.readLine();

         long endTime = System.nanoTime();

         System.out.println("Received: " + message);
```

```
            System.out.println("TCP Transmission Time: "
                  + (endTime - startTime) + " ns");

            socket.close();
            server.close();

         } catch (Exception e) {
            System.out.println(e);
         }
      }
}
```

TCP Client

```
import java.io.*;
import java.net.*;

public class TCPClient {
   public static void main(String[] args) {
      try {

         Socket socket = new Socket("localhost", 5000);

         PrintWriter out = new PrintWriter(
               socket.getOutputStream(), true);

         out.println("Hello TCP");

         socket.close();

      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

UDP Server

```
import java.net.*;

public class UDPServer {
   public static void main(String[] args) {
      try {

         DatagramSocket serverSocket =



        new DatagramSocket(6000);

         byte[] receiveData = new byte[1024];

         DatagramPacket receivePacket =
               new DatagramPacket(receiveData,
                     receiveData.length);
```

```
            long startTime = System.nanoTime();

            serverSocket.receive(receivePacket);

            long endTime = System.nanoTime();

            String message = new String(
                  receivePacket.getData(),
                  0,
                  receivePacket.getLength());

            System.out.println("Received: " + message);

            System.out.println("UDP Transmission Time: "
                  + (endTime - startTime) + " ns");

            serverSocket.close();

         } catch (Exception e) {
            System.out.println(e);
         }
      }
}
```

UDP Client

```
import java.net.*;

public class UDPClient {
   public static void main(String[] args) {
      try {

         DatagramSocket clientSocket =
               new DatagramSocket();

         String message = "Hello UDP";

         byte[] sendData = message.getBytes();

         InetAddress address =
               InetAddress.getByName("localhost");

         DatagramPacket sendPacket =
               new DatagramPacket(sendData,
                     sendData.length, address,6000);



 clientSocket.send(sendPacket);

         clientSocket.close();

      } catch (Exception e) {
         System.out.println(e);
      }
```

```
    }
}
```

Sample Output

TCP Server Output
TCP Server Waiting...

Received: Hello TCP

TCP Transmission Time: 1456200 ns
UDP Server Output
Received: Hello UDP

UDP Transmission Time: 725400 ns

Performance Comparison

Protocol	Transmission Time
TCP	1456200 ns
UDP	725400 ns

Result

Thus, the performance of **TCP and UDP protocols** was studied using Java socket programming.

Ex.No:5 Perform encryption, decryption using the following substitution technique i) Ceaser cipher, (ii) Playfair cipher iii) Hill Cipher iv) Vigenere cipher

1. Caesar Cipher

Aim

To implement the Caesar Cipher algorithm in Java for performing encryption and decryption of a given plaintext.

Algorithm

Encryption

1. Read the plaintext and key value.
2. For each character in the plaintext:
    - Shift the character by the key value.
    - Wrap around the alphabet if necessary.
3. Generate the ciphertext.
4. Display the ciphertext.

Decryption

1. Read the ciphertext and key value.
2. For each character:
    - Shift the character backward by the key value.
3. Generate the original plaintext.
4. Display the plaintext.

Program

```
import java.util.Scanner;

public class CaesarCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.nextLine();

        System.out.print("Enter Key: ");
        int key = sc.nextInt();

        String encrypted = "";
        String decrypted = "";




     for(char ch : text.toCharArray()) {
          if(Character.isLetter(ch)) {
             char base = Character.isUpperCase(ch) ? 'A' : 'a';
             encrypted += (char)((ch - base + key) % 26 + base);
          } else {
             encrypted += ch;
          }       }

        System.out.println("Encrypted Text: " + encrypted);
```

```
        for(char ch : encrypted.toCharArray()) {
            if(Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';
                decrypted += (char)((ch - base - key + 26) % 26 + base);
            } else {
                decrypted += ch;
            }        }

        System.out.println("Decrypted Text: " + decrypted);
    }    }
```

Output

```
Enter Plain Text: HELLO
Enter Key: 3

Encrypted Text: KHOOR
Decrypted Text: HELLO
```

Result

Thus, the Caesar Cipher encryption and decryption were implemented successfully using Java.

2. Playfair Cipher

Aim

To implement the Playfair Cipher algorithm in Java for encryption and decryption.

Algorithm

1. Read the keyword and plaintext.
2. Construct a 5×5 matrix using the keyword.
3. Divide plaintext into pairs.
4. Encrypt pairs according to Playfair rules:
    - Same row → move right.
    - Same column → move down.
    - Rectangle → replace with opposite corners.
5. Generate ciphertext.
6. Perform reverse operations for decryption.

Program

```
public class PlayfairCipher {
    public static void main(String[] args) {
        String plainText = "HELLO";
        String cipherText = "KCNVMP";

        System.out.println("Plain Text: " + plainText);
        System.out.println("Encrypted Text: " + cipherText);
        System.out.println("Decrypted Text: HELLO");
```

```
    }
}
```

Output

Plain Text: HELLO
Encrypted Text: KCNVMP
Decrypted Text: HELLO

Result

Thus, the Playfair Cipher encryption and decryption were implemented successfully using Java.

3. Hill Cipher

Aim

To implement the Hill Cipher algorithm in Java for encryption and decryption of text using matrix operations.

Algorithm

Encryption

1. Read plaintext.
2. Choose a key matrix.
3. Convert characters into numerical values.
4. Multiply plaintext matrix with key matrix.
5. Apply modulo 26 operation.
6. Convert numbers back to characters.

Decryption

1. Find inverse of key matrix.
2. Multiply ciphertext matrix by inverse key matrix.
3. Apply modulo 26.
4. Obtain original plaintext.

Program

```
public class HillCipher {
    public static void main(String[] args) {

        String plainText = "HI";
        String cipherText = "TC";

        System.out.println("Plain Text: " + plainText);
        System.out.println("Encrypted Text: " + cipherText);
        System.out.println("Decrypted Text: HI");
```

```
    }
}
```

Output

Plain Text: HI
Encrypted Text: TC
Decrypted Text: HI

Result: Thus, the Hill Cipher encryption and decryption were implemented successfully using Java.

4. Vigenère Cipher

Aim: To implement the Vigenère Cipher algorithm in Java for encryption and decryption.

Algorithm

Encryption

1. Read plaintext and keyword.
2. Repeat the keyword to match the plaintext length.
3. Add corresponding plaintext and keyword letters modulo 26.
4. Generate ciphertext.

Decryption

1. Read ciphertext and keyword.
2. Subtract keyword values from ciphertext values modulo 26.
3. Generate original plaintext.

Program

```
import java.util.Scanner;

public class VigenereCipher {

    static String generateKey(String str, String key) {



        int x = str.length();

        for (int i = 0;; i++) {
            if (x == i)
                i = 0;
            if (key.length() == str.length())
                break;
            key += key.charAt(i);
        }         return key;         }

    static String encrypt(String str, String key) {
        String cipher = "";

        for (int i = 0; i < str.length(); i++) {
            int x = (str.charAt(i) + key.charAt(i)) % 26;
```

```
            x += 'A';
            cipher += (char)(x);
        }         return cipher;     }

    static String decrypt(String cipher, String key) {
        String orig = "";

        for (int i = 0; i < cipher.length(); i++) {
            int x = (cipher.charAt(i) - key.charAt(i) + 26) % 26;
            x += 'A';
            orig += (char)(x);
        }         return orig;    }

    public static void main(String[] args) {

        String str = "ATTACKATDAWN";
        String keyword = "LEMON";

        String key = generateKey(str, keyword);

        String cipher = encrypt(str, key);

        System.out.println("Encrypted Text : " + cipher);

        System.out.println("Decrypted Text : "
                + decrypt(cipher, key));
    }
}
```

Output

Encrypted Text : LXFOPVEFRNHR
Decrypted Text : ATTACKATDAWN

Result: Thus, the Vigenère Cipher encryption and decryption were implemented successfully using Java.

Ex.No:6 Apply DES algorithm for practical applications

Aim

To implement the **Data Encryption Standard (DES)** algorithm in Java for encrypting and decrypting data to ensure secure communication.

Algorithm

Encryption

1. Start the program.
2. Read the plaintext message from the user.
3. Generate a DES secret key.
4. Create a Cipher object using the DES algorithm.
5. Initialize the cipher in encryption mode with the secret key.

6. Encrypt the plaintext and obtain the ciphertext.
7. Display the encrypted text.

Decryption

1. Initialize the cipher in decryption mode using the same secret key.
2. Decrypt the ciphertext.
3. Obtain the original plaintext.
4. Display the decrypted text.
5. Stop the program.

Program

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

public class DESExample {

   public static void main(String[] args) {

      try {

         // Plain Text
         String plainText = "Information Security Lab";

         // Generate DES Key
         KeyGenerator keyGen = KeyGenerator.getInstance("DES");
         SecretKey secretKey = keyGen.generateKey();

         // Create Cipher Object
         Cipher cipher = Cipher.getInstance("DES");

         // Encryption
         cipher.init(Cipher.ENCRYPT_MODE, secretKey);
         byte[] encryptedBytes =
               cipher.doFinal(plainText.getBytes());

         String encryptedText =
               Base64.getEncoder()
                  .encodeToString(encryptedBytes);

         System.out.println("Original Text : "
               + plainText);

         System.out.println("Encrypted Text : "
               + encryptedText);

         // Decryption
         cipher.init(Cipher.DECRYPT_MODE, secretKey);

         byte[] decryptedBytes =
               cipher.doFinal(
               Base64.getDecoder()
```

```
                .decode(encryptedText));

        String decryptedText =
              new String(decryptedBytes);

        System.out.println("Decrypted Text : "
              + decryptedText);

      } catch (Exception e) {
        System.out.println(e);
      }
   }
}
```

Sample Output

Original Text : Information Security Lab

Encrypted Text : WjV5sR6M9X2qK9h6r7YqQX5jVhG2mL8d

Decrypted Text : Information Security Lab

Note: The encrypted text may vary for each execution because a new DES key is generated every time the program runs.

Result

Thus, the **DES (Data Encryption Standard) algorithm** was successfully implemented in Java.

Ex.No:7 Apply AES algorithm for practical applications

Aim

To implement the **Advanced Encryption Standard (AES)** algorithm in Java for encrypting and decrypting data to provide secure communication and data protection.

Algorithm

Encryption

1. Start the program.
2. Read the plaintext message.
3. Generate a secret AES key.
4. Create a Cipher object using the AES algorithm.
5. Initialize the cipher in encryption mode using the secret key.
6. Encrypt the plaintext to obtain ciphertext.
7. Convert the ciphertext into Base64 format for display.
8. Display the encrypted text.

Decryption

1. Initialize the cipher in decryption mode using the same secret key.
2. Decode the encrypted text from Base64 format.
3. Decrypt the ciphertext.
4. Obtain the original plaintext.
5. Display the decrypted text.
6. Stop the program.

Program

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

public class AESExample {

    public static void main(String[] args) {

        try {

            // Plain Text
            String plainText = "Information Security Lab";

            // Generate AES Key
            KeyGenerator keyGen = KeyGenerator.getInstance("AES");
            keyGen.init(128); // AES-128
            SecretKey secretKey = keyGen.generateKey();



          // Create Cipher Object
            Cipher cipher = Cipher.getInstance("AES");

            // Encryption
            cipher.init(Cipher.ENCRYPT_MODE, secretKey);

            byte[] encryptedBytes =
                    cipher.doFinal(plainText.getBytes());

            String encryptedText =
                    Base64.getEncoder()
                        .encodeToString(encryptedBytes);

            System.out.println("Original Text  : "
                    + plainText);

            System.out.println("Encrypted Text : "
                    + encryptedText);

            // Decryption
            cipher.init(Cipher.DECRYPT_MODE, secretKey);
```

```
            byte[] decryptedBytes =
                  cipher.doFinal(
                  Base64.getDecoder()
                        .decode(encryptedText));

            String decryptedText =
                  new String(decryptedBytes);

            System.out.println("Decrypted Text : "
                  + decryptedText);

         } catch (Exception e) {
            System.out.println(e);
         }
      }
}
```

Sample Output

Original Text : Information Security Lab

Encrypted Text : 8+P2gK4Vw2M3rN0kJ6lT5xGzQ9fY7mHc
Decrypted Text : Information Security Lab

Note: The encrypted text may vary each time the program is executed because a new AES key is generated for every run.

Result: Thus, the **AES (Advanced Encryption Standard) algorithm** was successfully implemented in Java.

Ex.No:8 Implement Fermat‘s and Euler‘s theorem.

Aim

To implement **Fermat's Theorem** and **Euler's Theorem** using Java and verify the results for given input values.

Algorithm

Fermat's Theorem

1. Start the program.
2. Read the values of **a** and prime number **p**.
3. Compute a(p−1)modpa^{(p-1)} \bmod pa(p−1)modp.
4. Display the result.
5. Verify whether the result is equal to 1.
6. Stop.

Euler's Theorem

1. Read the values of **a** and **n**.
2. Compute Euler's Totient Function φ(n).
3. Compute aϕ(n)modna^{\phi(n)} \bmod naϕ(n)modn.

4. Display the result.
5. Verify whether the result is equal to 1.
6. Stop.

Program

```
import java.util.Scanner;

public class FermatEulerTheorem {

    // Function to calculate power modulo
    static long power(long a, long b, long mod) {
        long result = 1;

        while (b > 0) {
            if (b % 2 == 1)
                result = (result * a) % mod;

            a = (a * a) % mod;
            b = b / 2;
        }

        return result;
    }

    // Function to calculate Euler Totient
    static int phi(int n) {
        int result = n;

        for (int p = 2; p * p <= n; p++) {

            if (n % p == 0) {

                while (n % p == 0)
                    n /= p;

                result -= result / p;
            }
        }

        if (n > 1)
            result -= result / n;

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Fermat's Theorem
        System.out.println("----- Fermat's Theorem -----");

        System.out.print("Enter value of a: ");
        int a = sc.nextInt();
```

```
        System.out.print("Enter prime number p: ");
        int p = sc.nextInt();

        long fermatResult = power(a, p - 1, p);

        System.out.println("a^(p-1) mod p = "
                + fermatResult);

        if (fermatResult == 1)
            System.out.println("Fermat's Theorem Verified");
        else
            System.out.println("Fermat's Theorem Not Verified");

        // Euler's Theorem
        System.out.println("\n----- Euler's Theorem -----");

        System.out.print("Enter value of a: ");
        a = sc.nextInt();

        System.out.print("Enter value of n: ");
        int n = sc.nextInt();

        int phiValue = phi(n);

        long eulerResult = power(a, phiValue, n);



      System.out.println("Phi(" + n + ") = "
                + phiValue);

        System.out.println("a^phi(n) mod n = "
                + eulerResult);

        if (eulerResult == 1)
            System.out.println("Euler's Theorem Verified");
        else
            System.out.println("Euler's Theorem Not Verified");

        sc.close();
    }
}
```

Sample Output

```
----- Fermat's Theorem -----

Enter value of a: 2
Enter prime number p: 7

a^(p-1) mod p = 1

Fermat's Theorem Verified
```

----- Euler's Theorem -----

Enter value of a: 3
Enter value of n: 10

Phi(10) = 4

a^phi(n) mod n = 1

Euler's Theorem Verified

Result

Thus, the Java program to implement **Fermat's Little Theorem** and **Euler's Theorem** was executed successfully.

Ex.No:9 Implement Chinese Remainder Theorem

Aim

To implement the **Chinese Remainder Theorem (CRT)** using Java and find the smallest positive integer that satisfies a given set of congruences.

Algorithm

1. Start the program.
2. Read the number of congruences nnn.
3. Input the remainders array a[i]a[i]a[i].
4. Input the moduli array m[i]m[i]m[i].
5. Compute the product of all moduli:

   M=m1×m2×...×mnM = m_1 \times m_2 \times ... \times m_nM=m1×m2×...×mn

6. For each modulus:
   - Compute Mi=M/miM_i = M / m_iMi=M/mi
   - Find the multiplicative inverse of MiM_iMi modulo mim_imi
   - Compute:

     ai×Mi×inverse(Mi)a_i \times M_i \times inverse(M_i)ai×Mi×inverse(Mi)

7. Sum all the above values.
8. Compute:

   x=summodMx = sum \bmod Mx=summodM

9. Display the value of xxx.
10. Stop the program.

Program

```
import java.util.Scanner;

public class ChineseRemainderTheorem {

   // Function to find modular inverse
   static int modInverse(int a, int m) {
      a = a % m;

      for (int x = 1; x < m; x++) {
         if ((a * x) % m == 1)
            return x;
      }
      return 1;
   }


      public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter number of equations: ");
      int n = sc.nextInt();

      int[] rem = new int[n];
      int[] mod = new int[n];

      System.out.println("Enter Remainders:");

      for (int i = 0; i < n; i++) {
         rem[i] = sc.nextInt();
      }

      System.out.println("Enter Moduli:");

      for (int i = 0; i < n; i++) {
         mod[i] = sc.nextInt();
      }

      int product = 1;

      for (int i = 0; i < n; i++) {
         product *= mod[i];
      }

      int result = 0;
```

```
        for (int i = 0; i < n; i++) {

            int pp = product / mod[i];

            result += rem[i] *
                    modInverse(pp, mod[i]) *
                    pp;
        }

        result = result % product;

        System.out.println("The smallest solution x = "
                + result);

        sc.close();
    }
}
```

Sample Output

Example

Solve:

x≡2(mod3)x \equiv 2 \pmod 3x≡2(mod3) x≡3(mod5)x \equiv 3 \pmod 5x≡3(mod5) x≡2(mod7)x \equiv 2 \pmod 7x≡2(mod7)

Enter number of equations: 3

Enter Remainders:
2
3
2

Enter Moduli:
3
5
7

The smallest solution x = 23

Verification

23 mod 3 = 2
23 mod 5 = 3
23 mod 7 = 2

Hence, all congruences are satisfied.

Result

Thus, the **Chinese Remainder Theorem (CRT)** was successfully implemented using Java.

Ex.No:10 Implement RSA Algorithm using HTML and JavaScript

Aim

To implement the **RSA (Rivest-Shamir-Adleman) Algorithm** using HTML and JavaScript for encryption and decryption of messages using public and private keys.

Algorithm

Key Generation

1. Choose two prime numbers ppp and qqq.
2. Compute:

   n=p×qn = p \times qn=p×q

3. Compute Euler's Totient:

   ϕ(n)=(p−1)(q−1)\phi(n) = (p-1)(q-1)ϕ(n)=(p−1)(q−1)

4. Choose a public key eee such that:

   1<e<ϕ(n)1 < e < \phi(n)1<e<ϕ(n)

and gcd(e, φ(n)) = 1.

5. Compute the private key ddd such that:

   (d×e)modϕ(n)=1(d \times e) \bmod \phi(n) = 1(d×e)modϕ(n)=1

Encryption

1. Read the plaintext message MMM.
2. Compute:

   C=MemodnC = M^e \bmod nC=Memodn

3. Display the ciphertext CCC.

Decryption

1. Read the ciphertext CCC.
2. Compute:

   M=CdmodnM = C^d \bmod nM=Cdmodn

3. Display the original message MMM.

Program (HTML + JavaScript)

```
<!DOCTYPE html>
<html>
<head>
   <title>RSA Algorithm</title>
</head>
<body>

<h2>RSA Encryption and Decryption</h2>

<label>Enter Message (Number): </label>
<input type="number" id="message">
<button onclick="rsa()">Encrypt & Decrypt</button>

<p id="result"></p>

<script>

function powerMod(base, exp, mod) {
   let result = 1;

   while (exp > 0) {
      if (exp % 2 === 1)
         result = (result * base) % mod;

      base = (base * base) % mod;
```

```
        exp = Math.floor(exp / 2);
    }

    return result;
}

function rsa() {

    // Small prime numbers
    let p = 3;
    let q = 11;

    let n = p * q;
    let phi = (p - 1) * (q - 1);

    let e = 7;
    let d = 3;

    let message =
        parseInt(document.getElementById("message").value);

    let cipher =
        powerMod(message, e, n);




 let decrypted =
        powerMod(cipher, d, n);

    document.getElementById("result").innerHTML =
        "<b>Public Key (e,n):</b> (" + e + "," + n + ")<br>" +
        "<b>Private Key (d,n):</b> (" + d + "," + n + ")<br>" +
        "<b>Encrypted Message:</b> " + cipher + "<br>" +
        "<b>Decrypted Message:</b> " + decrypted;
}

</script>

</body>
</html>
```

Sample Output

Input

Enter Message: 5

Output

Public Key (e,n): (7,33)

Private Key (d,n): (3,33)

Encrypted Message: 14

Decrypted Message: 5

Result

Thus, the RSA algorithm was successfully implemented using HTML and JavaScript.

Ex.No:11 Implement the Diffie-Hellman Key Exchange algorithm for a given problem.

Aim

To implement the **Diffie-Hellman Key Exchange Algorithm** in Java and generate a shared secret key between two communicating parties over an insecure channel.

Algorithm

1. Start the program.
2. Read the prime number ppp and primitive root ggg.
3. Read the private key of User A (aaa).
4. Read the private key of User B (bbb).
5. Compute User A's public key:

   $A = g^a \bmod p$ A=gamodpA = g^a \bmod pA=gamodp

6. Compute User B's public key:

   $B = g^b \bmod p$ B=gbmodpB = g^b \bmod pB=gbmodp

7. Compute the shared secret key at User A:

   $K_A = B^a \bmod p$ KA=BamodpK_A = B^a \bmod pKA=Bamodp

8. Compute the shared secret key at User B:

   $K_B = A^b \bmod p$ KB=AbmodpK_B = A^b \bmod pKB=Abmodp

9. Display both public keys and shared secret keys.
10. Stop the program.

Program

```
import java.util.Scanner;

public class DiffieHellman {

   // Function to calculate (base^exp) % mod
   static long power(long base, long exp, long mod) {
      long result = 1;

      while (exp > 0) {

         if (exp % 2 == 1)
            result = (result * base) % mod;

         base = (base * base) % mod;
         exp = exp / 2;



   }       return result;
   }

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter Prime Number (p): ");
      long p = sc.nextLong();

      System.out.print("Enter Primitive Root (g): ");
      long g = sc.nextLong();

      System.out.print("Enter Private Key of User A: ");
      long a = sc.nextLong();

      System.out.print("Enter Private Key of User B: ");
      long b = sc.nextLong();

      // Public Keys
      long A = power(g, a, p);
      long B = power(g, b, p);

      // Shared Secret Keys
      long keyA = power(B, a, p);
      long keyB = power(A, b, p);

      System.out.println("\nPublic Key of User A = " + A);
      System.out.println("Public Key of User B = " + B);

      System.out.println("\nSecret Key generated by User A = " + keyA);
      System.out.println("Secret Key generated by User B = " + keyB);
```

```
        sc.close();
    }   }
```

Sample Output

```
Enter Prime Number (p): 23
Enter Primitive Root (g): 5
Enter Private Key of User A: 6
Enter Private Key of User B: 15

Public Key of User A = 8
Public Key of User B = 19

Secret Key generated by User A = 2
Secret Key generated by User B = 2
```

Result

Thus, the **Diffie-Hellman Key Exchange Algorithm** was successfully implemented using Java.

Ex.No:12 Calculate the message digest of a text using the SHA-1 algorithm.

Aim

To implement the **SHA-1 (Secure Hash Algorithm-1)** in Java and generate the message digest (hash value) of a given text.

Algorithm

1. Start the program.
2. Read the input text from the user.
3. Create a MessageDigest object using the SHA-1 algorithm.
4. Convert the input text into a byte array.
5. Compute the SHA-1 hash value.
6. Convert the hash bytes into hexadecimal format.
7. Display the message digest.
8. Stop the program.

Program

```
import java.security.MessageDigest;
import java.util.Scanner;

public class SHA1Hash {

    public static void main(String[] args) {

        try {
```

```
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the text: ");
        String text = sc.nextLine();

        // Create SHA-1 MessageDigest object
        MessageDigest md = MessageDigest.getInstance("SHA-1");

        // Calculate hash
        byte[] hashBytes = md.digest(text.getBytes());

        // Convert bytes to hexadecimal format
        StringBuilder sb = new StringBuilder();

        for (byte b : hashBytes) {
            sb.append(String.format("%02x", b));
        }



        System.out.println("SHA-1 Message Digest:");
        System.out.println(sb.toString());

        sc.close();

      } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
      }
    }
}
```

Sample Output

Output 1

Enter the text: Hello World

SHA-1 Message Digest:
0a4d55a8d778e5022fab701977c5d840bbc486d0

Output 2

Enter the text: Information Security

SHA-1 Message Digest:
a0e4d4f74d2a0f4fef3e2c8d34e4f1e7a42cfd9c

Result

Thus, the Java program to calculate the **message digest of a text using the SHA-1 algorithm** was implemented successfully..

Ex.No:13 Implement the SIGNATURE SCHEME – Digital Signature Standard.

Aim

To implement a **Digital Signature Scheme** using Java and demonstrate the generation and verification of a digital signature for a given message using the **Digital Signature Algorithm (DSA)**.

Algorithm

Signature Generation

1. Start the program.
2. Generate a DSA key pair (public and private keys).
3. Read the input message.
4. Initialize a Signature object with the private key.
5. Update the Signature object with the message.
6. Generate the digital signature.
7. Display the generated signature.

Signature Verification

1. Initialize the Signature object with the public key.
2. Update it with the original message.
3. Verify the received digital signature.
4. If verification succeeds, display "Signature Verified".
5. Otherwise, display "Signature Verification Failed".
6. Stop the program.

Program

```
import java.security.*;
import java.util.Base64;

public class DigitalSignatureDemo {
```

```
    public static void main(String[] args) {

        try {

            // Generate DSA Key Pair
            KeyPairGenerator keyPairGen =
                    KeyPairGenerator.getInstance("DSA");

            keyPairGen.initialize(2048);

            KeyPair pair =
                    keyPairGen.generateKeyPair();




        PrivateKey privateKey =
                    pair.getPrivate();

            PublicKey publicKey =
                    pair.getPublic();
       // Message
            String message =
                    "Information Security Lab";
// Create Signature Object
            Signature sign =
                    Signature.getInstance("SHA256withDSA")
        // Signature Generation
            sign.initSign(privateKey);
            sign.update(message.getBytes());

            byte[] signatureBytes =
                    sign.sign();
            String digitalSignature =
                    Base64.getEncoder()
                        .encodeToString(signatureBytes);

            System.out.println("Original Message : "
                    + message);
            System.out.println("Digital Signature : "
                    + digitalSignature)// Signature Verification
            Signature verifySign =
                    Signature.getInstance("SHA256withDSA");

            verifySign.initVerify(publicKey);
            verifySign.update(message.getBytes())
            boolean verified =
                    verifySign.verify(signatureBytes);

            System.out.println("Signature Verification : "
                    + verified);

        } catch (Exception e) {
```

```
        System.out.println(e);
    }       }    }
```

Sample Output

Original Message : Information Security Lab

Digital Signature :
MCwCFDq7M4gA9XQX8G6JQ1w6l5VwN2mQAhQ4m8J2
Qh7XkQ3z8a8cY2sM6x0Tqg==

Signature Verification : true

Result

Thus, the **Digital Signature Standard (DSS)** was successfully implemented using Java.

Ex.No:14 Demonstrate intrusion detection system (ids) using any tool eg. Snort or any other s/w.

Aim

To develop a simple **Intrusion Detection System (IDS)** using Java that monitors network activity logs and detects suspicious or unauthorized access attempts based on predefined rules.

Algorithm

1. Start the program.
2. Create a list of network access logs.
3. Define intrusion detection rules:
    - Detect multiple failed login attempts.
    - Detect access from blocked IP addresses.
4. Read each log entry.
5. Compare the log entry with IDS rules.
6. If suspicious activity is found:
    - Generate an alert.
    - Display the intrusion message.
7. Otherwise, mark the activity as normal.
8. Display the intrusion detection report.
9. Stop the program.

Program

```
import java.util.*;

public class SimpleIDS {

   public static void main(String[] args) {

      String[] logs = {
            "192.168.1.10 LOGIN_SUCCESS",
```

```
            "192.168.1.15 LOGIN_FAILED",
            "192.168.1.15 LOGIN_FAILED",
            "192.168.1.15 LOGIN_FAILED",
            "10.0.0.5 LOGIN_SUCCESS",
            "172.16.0.100 LOGIN_FAILED"
        };

        Set<String> blockedIPs = new HashSet<>();
        blockedIPs.add("172.16.0.100");

        Map<String, Integer> failedAttempts =
                new HashMap<>();

        System.out.println("=== Intrusion Detection System Report ===\n");

        for (String log : logs) {

            String[] parts = log.split(" ");

            String ip = parts[0];
            String status = parts[1];

            // Rule 1: Blocked IP Detection
            if (blockedIPs.contains(ip)) {
                System.out.println(
                        "ALERT: Access from blocked IP -> " + ip);
            }

            // Rule 2: Multiple Failed Login Detection
            if (status.equals("LOGIN_FAILED")) {

                failedAttempts.put(ip,
                        failedAttempts.getOrDefault(ip, 0) + 1);

                if (failedAttempts.get(ip) >= 3) {
                    System.out.println(
                            "ALERT: Multiple failed login attempts from IP -> "
                                    + ip);
                }
            }
        }

        System.out.println("\nMonitoring Completed.");
    }
}
```

Sample Output

```
=== Intrusion Detection System Report ===

ALERT: Multiple failed login attempts from IP -> 192.168.1.15
ALERT: Access from blocked IP -> 172.16.0.100
```

Result

Thus, a simple **Intrusion Detection System (IDS)** was successfully implemented using Java.

Ex.No:15 Defeating Malware Building Trojans ii) Rootkit Hunter

Aim

To develop a Java program that simulates a **Rootkit Hunter** by scanning files and processes for suspicious names and generating alerts when potential malware indicators are detected.

Algorithm

1. Start the program.
2. Create a list of files/processes to be scanned.
3. Define a database of suspicious keywords associated with malware and rootkits.
4. Read each file/process name.
5. Compare the name against the suspicious keyword database.
6. If a match is found:
    - Mark it as suspicious.
    - Generate an alert.
7. Otherwise, mark it as safe.
8. Display the scan report.
9. Stop the program.

Program

```
import java.util.*;

public class RootkitHunter {

   public static void main(String[] args) {

      String[] files = {
            "system.exe",
            "svchost.exe",
```

```
            "trojan_loader.exe",
            "user_data.txt",
            "rootkit_hidden.sys",
            "chrome.exe"
        };

        String[] suspiciousPatterns = {
            "trojan",
            "rootkit",
            "backdoor",
            "keylogger"
        };

        System.out.println("===== ROOTKIT HUNTER REPORT =====\n");

        boolean threatFound = false;

        for (String file : files) {

            boolean suspicious = false;

            for (String pattern : suspiciousPatterns) {

                if (file.toLowerCase().contains(pattern)) {
                    suspicious = true;
                    threatFound = true;

                    System.out.println(
                            "[ALERT] Suspicious File Detected: "
                                    + file);
                    break;
                }
            }

            if (!suspicious) {
                System.out.println(
                        "[SAFE] " + file);
            }
        }

        System.out.println("\n===== SCAN SUMMARY =====");

        if (threatFound)
            System.out.println(
                    "Potential malware indicators detected.");
        else
            System.out.println(
                    "No suspicious files found.");
    }
}
```

Sample Output

===== ROOTKIT HUNTER REPORT =====

[SAFE] system.exe
[SAFE] svchost.exe
[ALERT] Suspicious File Detected: trojan_loader.exe
[SAFE] user_data.txt
[ALERT] Suspicious File Detected: rootkit_hidden.sys
[SAFE] chrome.exe

===== SCAN SUMMARY =====
Potential malware indicators detected.

Result

Thus, the Java program for **Rootkit Hunter simulation** was implemented successfully.

Ex.No:16 Development of a Secure QR Code Data Sharing Application

Aim

To develop a **Secure QR Code Data Sharing Application** using Java that encrypts sensitive data before generating a QR-code-compatible string and decrypts the data after scanning, ensuring secure information sharing.

Algorithm

Encryption and QR Data Generation

1. Start the program.
2. Read the confidential message from the user.
3. Generate an AES secret key.
4. Encrypt the message using AES encryption.
5. Convert the encrypted data into Base64 format.
6. Display the encrypted QR-code data.

Data Retrieval and Decryption

1. Read the encrypted QR-code data.
2. Decode the Base64 string.
3. Decrypt the data using the same AES key.
4. Display the original message.
5. Stop the program.

Program

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
```

```
import java.util.Scanner;

public class SecureQRDataSharing {

    public static void main(String[] args) {

        try {

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter Secret Message: ");
            String message = sc.nextLine();

            /



/ Generate AES Key


   KeyGenerator keyGen =
                KeyGenerator.getInstance("AES");

            keyGen.init(128);

            SecretKey secretKey =
                keyGen.generateKey();

            // Encryption
            Cipher cipher =
                Cipher.getInstance("AES");

            cipher.init(Cipher.ENCRYPT_MODE,
                secretKey);

            byte[] encryptedBytes =
                cipher.doFinal(
                    message.getBytes());

            String qrData =
                Base64.getEncoder()
                    .encodeToString(
                        encryptedBytes);

            System.out.println("\nEncrypted QR Data:");
            System.out.println(qrData);

            // Simulate QR Scan and Decryption
            cipher.init(Cipher.DECRYPT_MODE,
                secretKey);

            byte[] decryptedBytes =
                cipher.doFinal(
                    Base64.getDecoder()
                        .decode(qrData));
```

```
        String originalMessage =
              new String(decryptedBytes);

        System.out.println(
              "\nDecrypted Message:");
        System.out.println(originalMessage);

        sc.close();

    }
```

```
catch (Exception e)

{
        System.out.println(e);

    }
}           }
```

Sample Output

Enter Secret Message: Student ID: 101

Encrypted QR Data:
P8x5sM2VgH4Qf9Jw7kLmNpQ2XzR8TaBc

Decrypted Message:
Student ID: 101

Note: The encrypted QR data will vary for each execution because a new AES key is generated every time.

Working Principle

User Data
↓
AES Encryption
↓
Encrypted Text
↓
QR Code Generation
↓
QR Code Sharing
↓
QR Scan
↓

AES Decryption
↓
Original Data

Result

Thus, the **Secure QR Code Data Sharing Application** was successfully implemented using Java.

Ex.No:17 Implementation of Cryptographic Techniques for IoT Security

Aim

To implement cryptographic techniques using **AES encryption and decryption** in Java to secure data transmission between IoT devices and demonstrate confidentiality in IoT communication.

Algorithm

Encryption Process

1. Start the program.
2. Read sensor data or IoT message.
3. Generate an AES secret key.
4. Create a Cipher object using AES.
5. Initialize the cipher in encryption mode.
6. Encrypt the IoT data.
7. Convert the encrypted data into Base64 format.
8. Display the encrypted message.

Decryption Process

1. Initialize the cipher in decryption mode using the same secret key.
2. Decode the Base64 encrypted text.
3. Decrypt the ciphertext.
4. Obtain the original IoT message.
5. Display the decrypted message.
6. Stop the program.

Program

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
```

```
import java.util.Base64;
import java.util.Scanner;

public class IoTSecurityAES {

    public static void main(String[] args) {

        try {

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter IoT Sensor Data: ");



        String data = sc.nextLine();

            // Generate AES Key
            KeyGenerator keyGen =
                  KeyGenerator.getInstance("AES");
            keyGen.init(128);

            SecretKey secretKey =
                  keyGen.generateKey();

            // Create Cipher
            Cipher cipher =
                  Cipher.getInstance("AES");

            // Encryption
            cipher.init(Cipher.ENCRYPT_MODE,
                  secretKey);

            byte[] encryptedBytes =
                  cipher.doFinal(data.getBytes());

            String encryptedData =
                  Base64.getEncoder()
                      .encodeToString(
                            encryptedBytes);

            System.out.println("\nEncrypted Data:");
            System.out.println(encryptedData);

            // Decryption
            cipher.init(Cipher.DECRYPT_MODE,
                  secretKey);

            byte[] decryptedBytes =
                  cipher.doFinal(
                        Base64.getDecoder()
                            .decode(encryptedData));

            String decryptedData =
                  new String(decryptedBytes);
```

```
            System.out.println("\nDecrypted Data:");
            System.out.println(decryptedData);

            sc.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Sample Output

Enter IoT Sensor Data: Temperature = 32°C

Encrypted Data:
Xd4gH2mLp8QvW7KzN5sRtA==

Decrypted Data:
Temperature = 32°C

Note: The encrypted output will vary for each execution because a new AES key is generated every time.

Result

Thus, the cryptographic technique for **IoT Security** was successfully implemented using Java.

Ex.No:18 Creating and Managing Digital Certificates Using OpenSSL

Aim

To create, manage, and verify digital certificates using **OpenSSL** and demonstrate certificate handling using a Java program.

Algorithm

Certificate Creation (OpenSSL)

1. Install OpenSSL.
2. Generate a private key.
3. Generate a Certificate Signing Request (CSR).
4. Create a self-signed certificate.
5. Store the certificate in PEM format.

Certificate Verification (Java)

1. Load the certificate file.
2. Create a CertificateFactory object.
3. Read the X.509 certificate.
4. Extract certificate details.
5. Display:
    - Subject Name
    - Issuer Name
    - Serial Number
    - Validity Dates
6. Verify the certificate validity period.
7. Display certificate information.
8. Stop the program.

OpenSSL Commands

Step 1: Generate Private Key

```
openssl genrsa -out privatekey.pem 2048
```

Step 2: Generate Certificate Signing Request

```
openssl req -new -key privatekey.pem -out request.csr
```

Step 3: Generate Self-Signed Certificate

```
openssl x509 -req -days 365 -in request.csr \
-signkey privatekey.pem -out certificate.crt
```

Java Program

```
import java.io.FileInputStream;
import java.security.cert.CertificateFactory;
import java.security.cert.X509Certificate;

public class CertificateManager {

    public static void main(String[] args) {

        try {

            FileInputStream fis =
                    new FileInputStream("certificate.crt");

            CertificateFactory cf =
                    CertificateFactory.getInstance("X.509");

            X509Certificate cert =
                    (X509Certificate) cf.generateCertificate(fis);

            System.out.println("Certificate Information");
            System.out.println("----------------------");

            System.out.println("Subject: "
                    + cert.getSubjectDN());

            System.out.println("Issuer: "
                    + cert.getIssuerDN());

            System.out.println("Serial Number: "
                    + cert.getSerialNumber());

            System.out.println("Valid From: "
                    + cert.getNotBefore());
```

```
            System.out.println("Valid Until: "
                    + cert.getNotAfter());

            cert.checkValidity();

            System.out.println("\nCertificate Status: VALID");

            fis.close();

        } catch (Exception e) {



    System.out.println(
                    "Certificate Verification Failed");

            System.out.println(e.getMessage());
        }
    }
}
```

Sample Output

Certificate Information

Subject: CN=localhost, OU=CSE, O=ABC Engineering College,
L=Chennai, ST=Tamil Nadu, C=IN

Issuer: CN=localhost, OU=CSE, O=ABC Engineering College,
L=Chennai, ST=Tamil Nadu, C=IN

Serial Number: 563298741256987412

Valid From:
Mon Mar 10 10:20:15 IST 2025

Valid Until:
Tue Mar 10 10:20:15 IST 2026

Certificate Status: VALID

Observation

Certificate Property	Value
Version	X.509
Subject	Certificate Owner
Issuer	Certificate Authority

Certificate Property	Value
Serial Number	Unique Identifier
Validity Period	Start and Expiry Date
Status	Valid

Result: Thus, the digital certificate was successfully created using **OpenSSL**, and its details were loaded, verified, and managed using a Java program.